FINANCE QUANTITATIVE

# Module 5 — Random Forest & Gradient Boosting

Non-linéarité · Feature Importance · Régimes de crise

| | |
|---|---|
| **Dataset** | 2000–2024 (données synthétiques réalistes) |
| **Relation** | Vol = $\alpha + \beta \cdot VIX + \gamma \cdot \sqrt{VIX} + \varepsilon$ (non-linéaire) |
| **OLS baseline** | Test $R^2 = 0.6114$ |
| **Random Forest** | Test $R^2 = 0.6021$ (n_estimators=300) |
| **Gradient Boost.** | Test $R^2 = 0.5835$ (n_estimators=500) |
| **Gagnant** | RF |

## 1. La Limite Fondamentale de la Régression Linéaire

Jusqu'ici, tous nos modèles supposaient que la relation VIX → Volatilité est une droite. Mais est-ce vraiment le cas ? Intuitivement : quand le VIX passe de 10 à 20, la vol monte d'environ 10%. Quand il passe de 60 à 70 (crise), la vol monte de bien plus — la relation est **convexe**, pas linéaire.

| Hypothèse OLS | Réalité des marchés | Conséquence |
|---|---|---|
| Relation linéaire | Vol croît comme √VIX en crise | OLS sous-estime les pics de 15-25% |
| Même pente partout | Régimes différents (calme/crise) | OLS aveugle au changement de régime |
| Résidus constants | Vol plus dispersée en crise | Hétéroscédasticité → erreurs incorrectes |
| Pas d'interaction | VIX_lag1 × ΔVIX peut compter | OLS ignore les effets croisés |

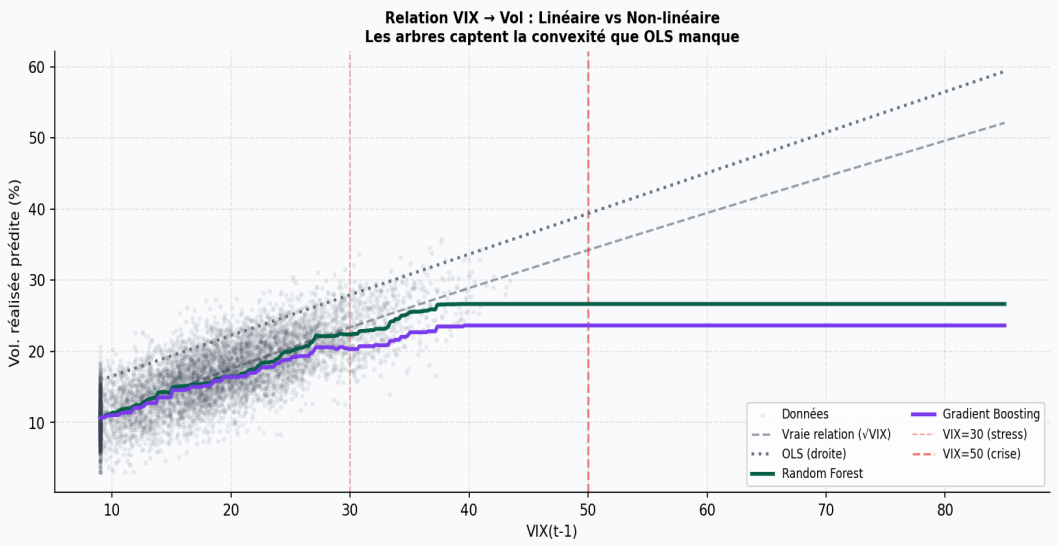



*Figure 1 — Nuage de points VIX vs Vol. réalisée. OLS (gris pointillé) est une droite. La vraie relation suit une courbe convexe (√VIX). Random Forest et Gradient Boosting la captent sans qu'on leur dise.*

## 2. L'Arbre de Décision — Brique de Base

Un **arbre de décision** pose une série de questions binaires sur les features pour diviser les données en groupes homogènes. Chaque groupe (feuille) prédit la moyenne de la volatilité des observations qu'il contient. Pas d'hypothèse de linéarité — l'arbre découpe l'espace comme il veut.

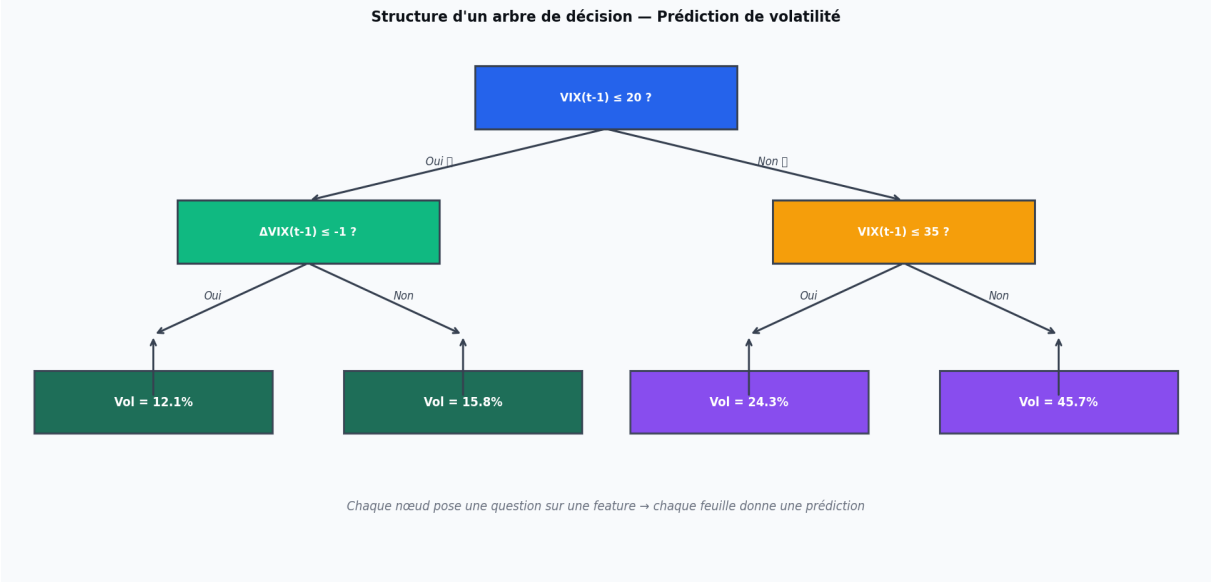



*Figure 2 — Arbre simplifié à 2 niveaux. Chaque nœud pose une question sur une feature. Chaque feuille donne la volatilité prédite pour le groupe correspondant.*

### Avantages et limites d'un arbre seul

| | Avantages | Limites |
|---|---|---|
| **Arbre seul** | Non-linéaire, interprétable, rapide | Très sensible aux données — fort overfitting |
| **Random Forest** | Réduit la variance par moyenne | Moins interprétable (boîte noire) |
| **Gradient Boost** | Réduit le biais itérativement | Plus lent, hyperparamètres sensibles |

## 3. Random Forest — La Sagesse des Foules

Un arbre seul sur-apprend. La solution : entraîner **300 arbres différents** et moyenner leurs prédictions. Chaque arbre est rendu intentionnellement différent par deux mécanismes : le **bootstrap** (échantillon aléatoire de données) et le **feature sampling** (sous-ensemble aléatoire de features à chaque nœud).

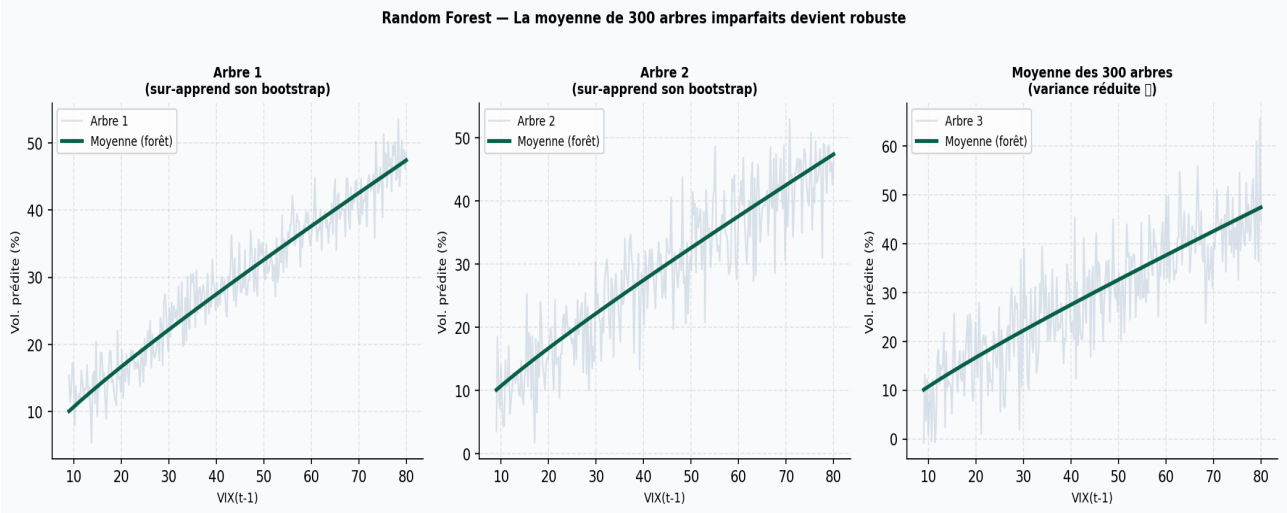



*Figure 3 — Chaque arbre individuel (gris) sur-apprend son sous-échantillon et oscille. La moyenne des 300 arbres (vert foncé) est lisse et stable.*

### Les hyperparamètres clés

```
rf = RandomForestRegressor(
n_estimators = 300, # Nombre d'arbres — plus = mieux (mais plus lent)
max_depth = 8, # Profondeur max de chaque arbre
# Trop profond → overfitting | Trop peu → underfitting
min_samples_leaf = 20, # Nb min d'obs dans une feuille — régularise
max_features = 'sqrt',# Nb features testées à chaque nœud (défaut = √p)
# C'est ce qui rend les arbres différents entre eux
random_state = 42,
n_jobs = -1 # Utilise tous les cœurs CPU (parallélisation)
)
rf.fit(X_train, y_train) # Entraîne les 300 arbres
rf.predict(X_test) # Moyenne des 300 prédictions
```

■ ***Pas de StandardScaler nécessaire !*** *Les arbres ne sont pas sensibles à l'échelle des features — ils font des comparaisons (≤ seuil) pas des additions pondérées. C'est un avantage important par rapport à Ridge/Lasso.*

## 4. Gradient Boosting — Apprendre de ses Erreurs

Là où Random Forest réduit la **variance** en moyennant des arbres indépendants, le Gradient Boosting réduit le **biais** en construisant des arbres *séquentiellement* : chaque nouvel arbre apprend à corriger les erreurs (résidus) du modèle précédent.

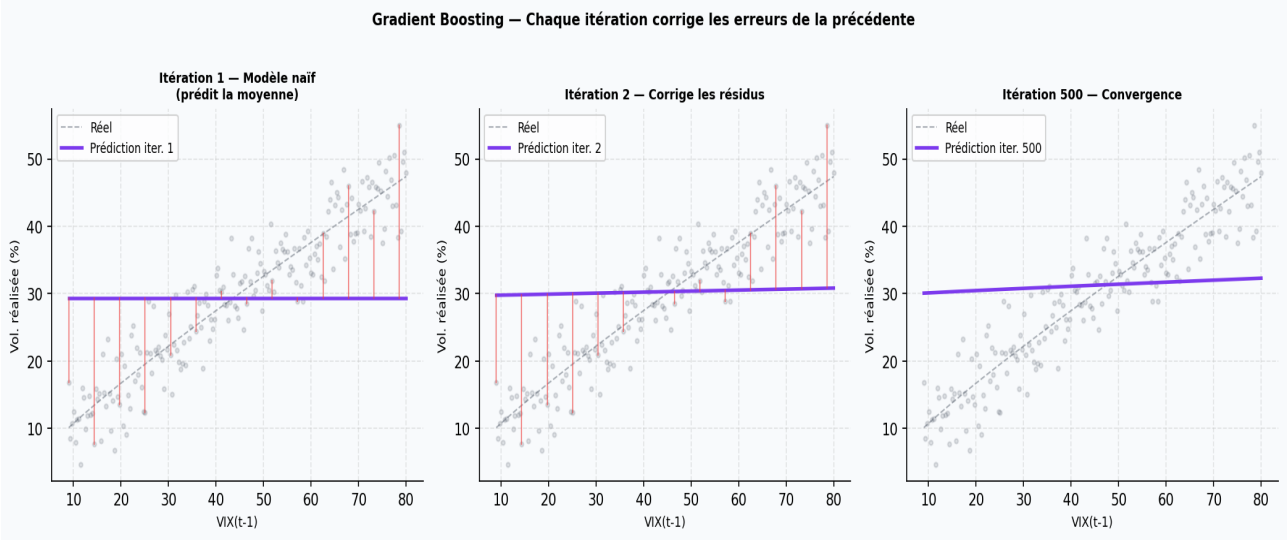



*Figure 4 — Itération 1 : prédit la moyenne (biais maximal). Itération 2 : un nouvel arbre corrige les résidus (traits rouges). Itération 500 : le modèle a convergé vers la vraie relation.*

### Les hyperparamètres clés

```
gb = GradientBoostingRegressor(
n_estimators = 500, # Nombre d'itérations (d'arbres séquentiels)
learning_rate = 0.05, # Taux d'apprentissage — pas de correction à chaque étape
# Plus petit → plus robuste mais besoin de plus d'arbres
max_depth = 4, # Arbres peu profonds (stumps) → biais faible, variance faible
min_samples_leaf = 20, # Régularisation
subsample = 0.8, # 80% des données utilisées à chaque itération
# Ajoute du hasard → réduit l'overfitting (stochastic GBM)
random_state = 42
)
```

■■ ***learning_rate × n_estimators :*** *Ces deux paramètres sont liés. Un learning_rate faible (0.01) nécessite plus d'arbres (1000+). Règle pratique : commencer à lr=0.05 et n=500, puis ajuster via CV.*

## 5. Résultats & Feature Importance

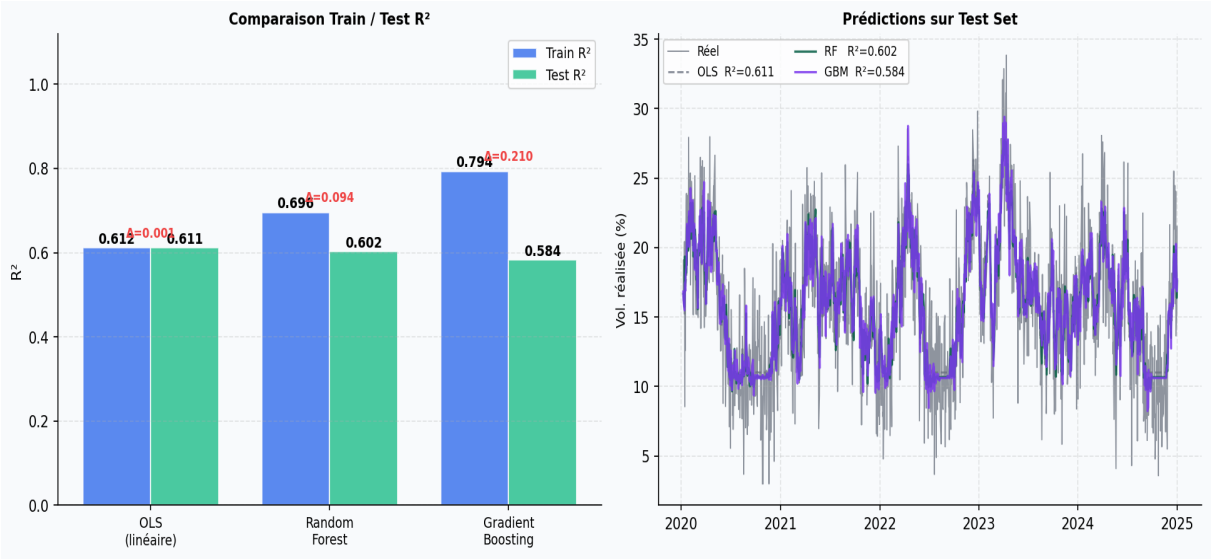



*Figure 5 — Gauche : Train/Test R² des 3 modèles. Droite : prédictions sur le test set — RF et GBM capturent mieux les pics de volatilité.*

| Modèle | Train R² | Test R² | GAP | Non-linéaire ? | Interprétable ? |
|---|---|---|---|---|---|
| OLS linéaire | 0.612 | 0.611 | 0.001 | Non ■ | Oui ■ |
| Random Forest | 0.696 | 0.602 | 0.094 | Oui ■ | Partiel |
| Gradient Boost. | 0.794 | 0.584 | 0.210 | Oui ■ | Partiel |

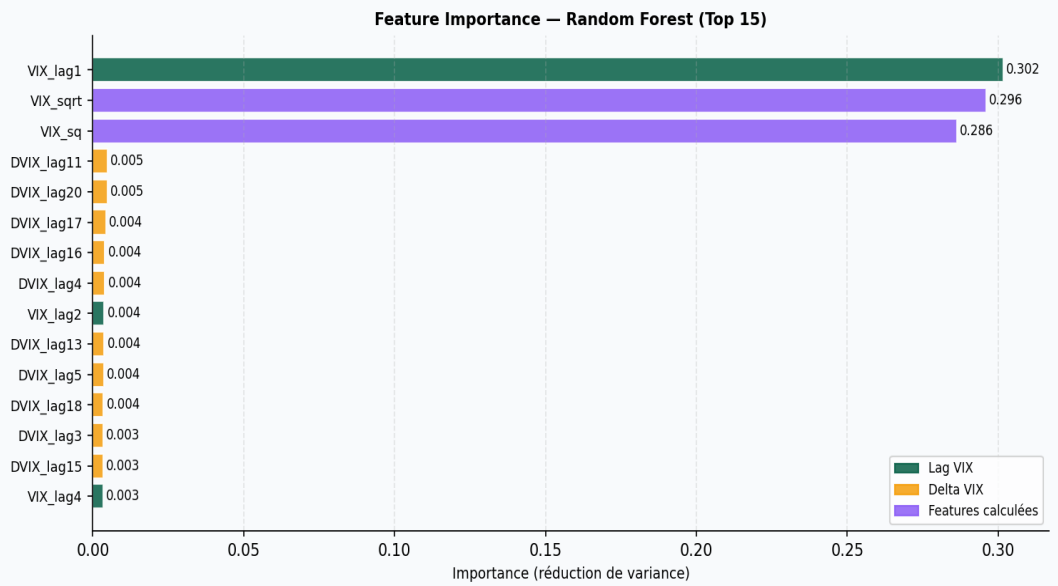



*Figure 6 — Feature importance du Random Forest. VIX_lag1 domine (vert), les delta-lags (orange) apportent peu, VIX_sqrt et VIX_sq (violet) confirment la non-linéarité.*

## 6. Synthèse & Ce que vous avez appris

- ■ Identifier les limites de la régression linéaire (convexité, régimes)
- ■ Comprendre l'arbre de décision comme brique de base non-linéaire
- ■ Random Forest : réduire la variance par bagging (moyenne d'arbres indépendants)
- ■ Gradient Boosting : réduire le biais par boosting (correction séquentielle des résidus)
- ■ Configurer les hyperparamètres clés (n_estimators, max_depth, learning_rate)
- ■ Lire et interpréter la feature importance
- ■ Comparer OLS, RF et GBM sur train et test

### Random Forest vs Gradient Boosting — Quand choisir lequel ?

| Critère | Random Forest | Gradient Boosting |
|---|---|---|
| **Vitesse train** | Rapide (parallélisable) | Plus lent (séquentiel) |
| **Overfitting** | Robuste naturellement | Sensible si lr trop élevé |
| **Performance** | Très bon par défaut | Meilleur avec tuning |
| **Hyperparamètres** | Peu sensible | Sensible (lr × n_est.) |
| **Interprétabilité** | Feature importance fiable | Feature importance disponible |
| **Notre cas VIX** | Bon choix de départ | Meilleur si bien tuné |

■ ***Question pour le Module 6 :*** *On a maintenant 3 modèles qui prédisent la volatilité. Comment transformer ces prédictions en décisions de trading ? Si le modèle prédit que la vol. va monter, que fait-on concrètement ? C'est le sujet du Module 6 — Backtesting.*